



**UNIVERSITI
MALAYA**

WID3011 DEEP LEARNING

SEMESTER 2, 2025/2026

MID-TERM BREAK ASSIGNMENT:

CNN-BASED IMAGE CLASSIFIER

FOR MALAYSIAN PRODUCT RECOGNITION

LECTURER:

PROFESOR MADYA TS. DR. AZNUL QALID BIN MD SABRI

NAME	MATRIC NUMBER
OOI RUI ZHE	23004947

TABLE OF CONTENTS

Part A: Dataset Preparation	4
Dataset Source & Structure Overview	4
Data Cleaning & Preprocessing	4
Data Exploration	5
Data Partitioning & Splitting	5
Data Imbalance Handling	6
The classed weight of the five classes are computed as followed:	6
Data Augmentation Strategy	6
Part B: CNN Architecture Design	8
Architectural Design	8
Architecture Summary Table	8
Parameter Count & Layer Shape	9
Receptive Field Analysis	10
Training Configuration and Regularisation Techniques	11
Part C: Transfer Learning Comparison	12
2-Stage Training Strategy	12
Quantitative Comparison	12
Convergence Speed Analysis	13
Analysis Learning Curves	14
Part D: Analysis & Reflection	16
Misclassified Sample Evaluation	16
Proposed Model Enhancement	18
Application Considerations in SME Context	20
References	21
Appendix	22

Part A: Dataset Preparation

Collect or curate a dataset of ≥ 5 Malaysian product/food categories (≥ 100 images each). Apply data augmentation. Document your class-imbalance strategy.

Dataset Source & Structure Overview

This project uses the IEEE [DataPort MF-150 Multilabel Malaysian Foods Dataset] (Tahir & Loo, 2020) which is available in IEEE DataPort Platform, a globally accessible data platform. The dataset was introduced by Ghalib Ahmed Tahir and Loo Chu Kiong in 2020 and is designed for food image analysis and ingredient detection tasks.

This dataset consists of a large collection of Malaysian food images covering 150 food categories of local dishes and each image is annotated with multiple ingredient labels. However, this project utilises only the food images and their corresponding category uses for a single-label classification task. Each image is assigned to one dominant food category based on its folder label, while the ingredient annotations are not used.

The list of the 5 selected classes are as followed:

Class ID	Dish	Total Images
2	Murtabak	249
84	Satay	200
133	Chicken Bak Kut Teh	182
8	Char Kuey Teow	178
82	Hong Kong Wonton Noodles	170

Five classes are used instead of more so the model can learn better with the limited data, while still having enough variety in the dataset.

Data Cleaning & Preprocessing

The image dataset is actually pre-split by the original authors into train and testing folders, with each class folders nested as numeric IDs inside each (1–150). To map these IDs to human-readable dish names, the folder IDs were joined against foldernames.csv provided with the dataset so that both its numeric label and its display name are identified. Since the

dataset is pre-split, the train and test images for each class were pooled into a single per-class collection. It will be then repartitioned into 70/15/15 train/validation/test sets.

Data Exploration

The five selected classes have a total of 979 images, where Murtabak has 249 total images, Satay has 200 images, Chicken Bak Kut Teh has 182 images, Char Kuey Teow has 178 images and Hong Kong wonton Noodles has a total of 170 images. The distribution is moderately uneven but not severe, and the class imbalance will be handled later.

All these five dishes are popular Malaysian local cuisines, where Char Kuey Teow and Hong Kong Wonton Noodles are both noodle dishes, Chicken Bak Kut Teh is a soup-based dish. Satay and Murtabak are also widely recognised as popular street foods in Malaysia. These variations in visual characteristics across the dishes make the dataset suitable for training a custom CNN model to recognise Malaysian food categories efficiently.

Data Partitioning & Splitting

The pooled dataset of 979 images was partitioned using a stratified 70/15/15 split with random_state=42 for reproducibility. The result of splitting are as followed:

	Murtabak	Satay	Chicken Bak Kut Teh (Ayam Bak Kut Teh)	Char Kuey Teow	Hong Kong Wonton Noodles	Total
Train	174	140	127	125	119	685
Validation	38	30	28	26	25	147
Testing	37	30	27	27	26	147

The training set is used to train the custom CNN model, the validation set is used during training to monitor model performance and tune hyperparameters, while the testing set is used after training is complete to evaluate the final performance of the model on unseen data.

Data Imbalance Handling

Class-weighted cross-entropy loss was selected as the data imbalance handling strategy because other two approaches like undersampling would lead to unnecessary loss of valuable data, while oversampling through duplication may cause the model to overfit.

In this approach, the class weights were computed using sklearn's `compute_class_weight` function with the balanced option as followed:

$$\text{Class weights} = \frac{n_{\text{samples}}}{n_{\text{classes}} \times n_{\text{samples in class}}}$$

This assigns higher weights to minority classes and lower weights to majority classes, helping to reduce bias without changing the original dataset distribution.

The classed weight of the five classes are computed as followed:

Index	Class ID	Dish Name	Train_column	Weight
0	2	Murtabak	174	0.787
1	84	Satay	140	0.979
3	133	Chicken bak kut teh (Ayam bak kut teh)	127	1.079
4	8	Char kuey teow	125	1.096
82	82	Hong Kong Wonton noodles	119	1.151

The imbalance ratio (max/min count) on the training split is 1.46, confirming that the imbalance is mild and class-weighted loss is sufficient. No resampling was applied at the dataset level.

Data Augmentation Strategy

Several augmentation strategies are applied on the training split data as followed:

Transform	Parameters	Justification
Resize	(256, 256)	Provides margin for random crop step.
RandomResizedCrop	size=224, scale=(0.75, 1.0)	Simulates framing variation in food photos.
RandomHorizontalFlip	p=0.5	Food appearance is largely orientation-invariant from above.
RandomRotation	$\pm 15^\circ$	Phone-camera shots are rarely perfectly level.
ColorJitter	brightness/contrast/saturation = 0.2	Builds invariance to lighting and colour temperature.
Normalize	ImageNet mean/std	Required for ResNet-50; applied uniformly for fair comparison with the custom CNN.

Augmentation Design Constraints:

- Random vertical flip is not used because many dishes have a fixed or canonical orientation
- Aggressive cropping with very small scale values (scale<0.5) is avoided as it may remove important parts of the dish from the image
- Strong colour jitter is not applied since it can distort colour features that help distinguish similar dishes

Part B: CNN Architecture Design

Design and implement a custom CNN from scratch (min. 4 conv layers). Report parameter count, layer shapes, and receptive field analysis for each convolutional block.

Architectural Design

Given the input images set are 224×224 , a 5-stage convolutional CNN (six convolutional layers in total) was implemented from scratch. Each stage follows the standard pattern Conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool.

Each ConvBlock contains:

- a 3×3 Conv2d layer with same padding to preserve spatial dimensions
- Batch Normalisation to stabilise training and improve convergence
- a ReLU activation to introduce non-linearity
- a 2×2 Max Pooling layer with stride 2 is used to reduce the spatial resolution and gradually extract higher-level features.

The first two stages follow the VGG-style two-convolution pattern, where two 3×3 convolutions are stacked at the same spatial resolution. This achieves the same effective receptive field as a single 5×5 convolution, but with fewer parameters and an additional non-linearity. The early stages have already captured the fine-grained texture features that benefit most from additional refinement, and the deeper stages use only a single conv per stage because the number of parameters grows quadratically with the number of channels.

The network has six convolutional layers. The 224×224 input is reduced to a $14 \times 14 \times 256$ feature map through four $2 \times$ downsampling operations. The classifier head follows the pattern Flatten $\rightarrow$ Linear($50176 \rightarrow 256$) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear($256 \rightarrow 5$). Dropout is used to regularise the dense layer and to prevent overfitting on the small dataset.

Architecture Summary Table

Stage	Layers	Output Channels	Spatial Size
Input	—	3	224×224

Stage 1	Conv $3 \times 3 \rightarrow$ BN $\rightarrow$ ReLU $\times 2$, MaxPool 2×2	32	112×112
Stage 2	Conv $3 \times 3 \rightarrow$ BN $\rightarrow$ ReLU $\times 2$, MaxPool 2×2	64	56×56
Stage 3	Conv $3 \times 3 \rightarrow$ BN $\rightarrow$ ReLU, MaxPool 2×2	128	28×28
Stage 4	Conv $3 \times 3 \rightarrow$ BN $\rightarrow$ ReLU, MaxPool 2×2	256	14×14
Head	Flatten $\rightarrow$ FC 256 $\rightarrow$ ReLU $\rightarrow$ Dropout 0.5 $\rightarrow$ FC 5	—	—

Parameter Count & Layer Shape

Parameter Count:

Parameters were counted using the formulas $k \times k \times C_{in} \times C_{out}$ (for convolution), $2 \times C$ (for BatchNorm), and $in \times out + out$ (for fully connected layers), and then double-checked using PyTorch's `parameters()` function. The output are as followed:

Layer	Parameters
-----	-----
Stage1 Conv1 (3 -> 32)	928
Stage1 Conv2 (32 -> 32)	9,280
Stage2 Conv1 (32 -> 64)	18,560
Stage2 Conv2 (64 -> 64)	36,992
Stage3 Conv (64 -> 128)	73,984
Stage4 Conv (128 -> 256)	295,424
FC1 (50176 -> 256)	12,845,312
FC2 (256 -> n_classes)	1,285
-----	-----
Total (manual count)	13,281,765
Total (PyTorch count)	13,281,765

Feature-map shapes through the network:

The feature map shapes are calculated using the formula $(m+2p-k)/s+1$ for each layer. Convolution layers keep the spatial size the same when padding = 1 and stride = 1, but they

change the number of channels. Max-pooling layers reduce the height and width by half. This is repeated from the input to find the shape at each stage of the network.

Feature map shapes through the network:

	layer	shape
	Input	224x224x3
Stage1	Conv1 (3x3,same)	224x224x32
Stage1	Conv2 (3x3,same)	224x224x32
Stage1	MaxPool (2x2,s=2)	112x112x32
Stage2	Conv1 (3x3,same)	112x112x64
Stage2	Conv2 (3x3,same)	112x112x64
Stage2	MaxPool (2x2,s=2)	56x56x64
Stage3	Conv (3x3,same)	56x56x128
Stage3	MaxPool (2x2,s=2)	28x28x128
Stage4	Conv (3x3,same)	28x28x256
Stage4	MaxPool (2x2,s=2)	14x14x256

Receptive Field Analysis

The receptive field (RF) is the spatial region of the input image that influences a neuron at a given depth. The standard recurrence used is:

$$RF_{\ell} = RF_{\ell-1} + (k_{\ell} - 1) \cdot J_{\ell-1}, \quad J_{\ell} = J_{\ell-1} \cdot s_{\ell}$$

where k is kernel size, s is stride, and J (jump) is the cumulative stride. Starting from $RF_0 = 1$ and $J_0 = 1$ and applying the recurrence layer by layer produces the following table:

Receptive field after each layer:

	layer	kernel	stride	jump	receptive_field
Stage1	Conv1 (3x3,same)	3	1	1	3x3
Stage1	Conv2 (3x3,same)	3	1	1	5x5
Stage1	MaxPool (2x2,s=2)	2	2	2	6x6
Stage2	Conv1 (3x3,same)	3	1	2	10x10
Stage2	Conv2 (3x3,same)	3	1	2	14x14
Stage2	MaxPool (2x2,s=2)	2	2	4	16x16
Stage3	Conv (3x3,same)	3	1	4	24x24
Stage3	MaxPool (2x2,s=2)	2	2	8	28x28
Stage4	Conv (3x3,same)	3	1	8	44x44
Stage4	MaxPool (2x2,s=2)	2	2	16	52x52

Final RF at deepest conv: 52x52 (input is 224x224)

Coverage: ~23% of the input image

The final receptive field at the deepest layer is 52×52 , which is about 23% of the 224×224 input. The hierarchical progression interprets as follows:

- Stage 1 (RF ≈ 6): local edges, colour blobs, simple textures.
- Stage 2 (RF ≈ 16): small textures - grains of rice, surface of wonton skin.
- Stage 3 (RF ≈ 28): medium parts - a single satay stick, one fish ball.
- Stage 4 (RF ≈ 52): larger dish regions - clusters of garnish, plate sections.

This shows how CNNs learn from simple features to more complex patterns.

Training Configuration and Regularisation Techniques

The custom CNN was trained with the following configuration:

Hyperparameter / Component	Value
Optimiser	Adam, weight_decay = $1e-4$
Initial learning rate	3×10^{-4}
LR schedule	CosineAnnealingLR over the full epoch budget
Loss function	Cross-entropy with class weights from Part A
Batch size	32
Maximum epochs	50
Early stopping patience	10 epochs (on validation accuracy)
Random seed	42 (NumPy + PyTorch + scikit-learn)
Hardware	Google Colab T4 GPU (CUDA)

Regularisation techniques used:

- Data augmentation
 - provides label-preserving input variation.
- Dropout ($p = 0.5$) on the FC1 output
 - reduces co-adaptation and over-reliance on individual features.
- L2 weight decay ($1e-4$) via Adam
 - discourages excessively large weights.

Part C: Transfer Learning Comparison

Fine-tune a pre-trained ResNet-50 on the same dataset. Compare accuracy, training time, and convergence speed against your custom CNN. Provide a learning curve for both.

2-Stage Training Strategy

Following standard transfer-learning best practice, ResNet-50 was fine-tuned in two distinct stages:

- Stage 1: Head warm-up (frozen backbone)
 - All ResNet-50 pretrained backbone layers are frozen. Only the new randomly-initialised FC layer trains, with a relatively high learning rate of 1×10^{-3} , for up to 10 epochs. This allows the new classifier to settle into reasonable weights without disturbing the pretrained features
- Stage 2: Full fine-tuning (all layers unfrozen)
 - All layers are unfrozen and the entire network is trained end-to-end with a $10\times$ smaller learning rate of 1×10^{-4} for up to 20 epochs. The smaller LR ensures gradients do not destroy the carefully-learned ImageNet features while still allowing them to adapt to Malaysian food images.

Quantitative Comparison

Both models were trained on the same splits and transformations and evaluated on the held-out test set of 147 images. The comparison table below summarises the key metrics:

	Metric	Custom CNN	ResNet-50
0	Total parameters	13,281,765	23,518,277
1	Best validation accuracy	93.20%	100.00%
2	Test accuracy	92.52%	99.32%
3	Total training time (s)	380.3	265.2
4	Mean epoch time (s)	8.3	9.5
5	Epochs trained (early stopping)	46	28
6	Epochs to reach 70% val acc	8	1

Explanation: The test results show that ResNet-50 achieves near-perfect accuracy at 99.32%, significantly outperforming the Custom CNN, which only 92.52%. From here, the ResNet-50 generalises much better on unseen data and is more reliable for classification across all dish categories.

Convergence Speed Analysis

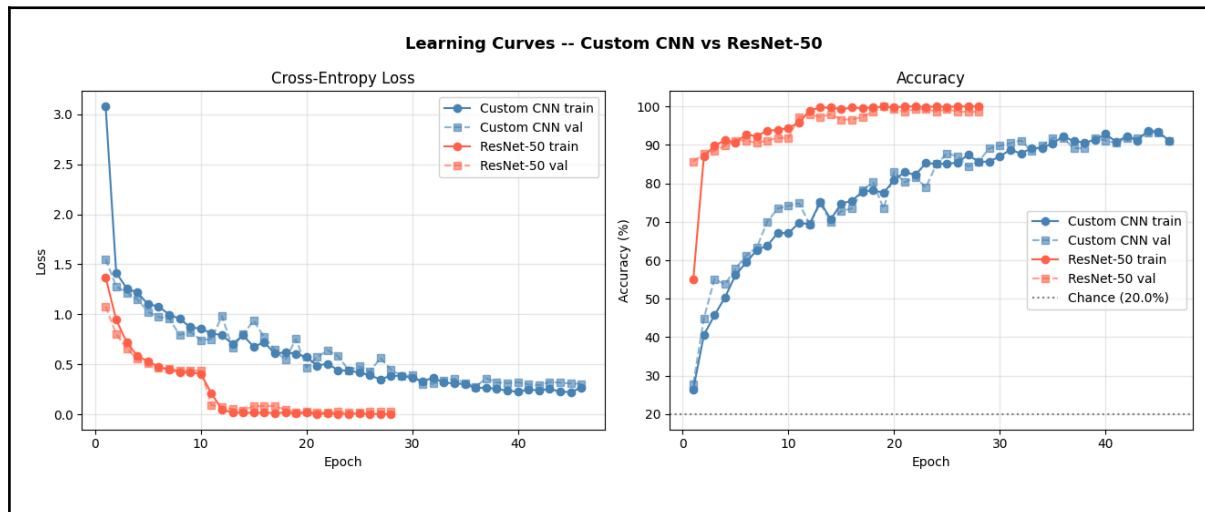
There is a clear and significant difference in convergence behaviour between the custom CNN and the fine-tuned ResNet-50 based on the

ResNet-50 model converges extremely quickly. During Stage 1 of head warm-up, it already achieves 85.71% validation accuracy in the first epoch, and reaches 91.84% within 10 epochs. After transitioning to Stage 2 (full fine-tuning), the model rapidly improves further, achieving 97.28% validation accuracy at the first epoch of fine-tuning, and reaching 100% validation accuracy by epoch 9. Overall, the model completes training in 28 total epochs with a combined training time of approximately 265 seconds.

Meanwhile, the custom CNN exhibits a much more gradual convergence. In Stage 1, it only achieves 26.42% and requires only 8 epochs to be able to reach 70% validation accuracy. It continues improving until it reaches its best validation accuracy of 93.20% at epoch 44. The training process is longer, stopping at epoch 46 due to early stopping, with total training time of 380.3 seconds.

ResNet-50 has faster convergence, fewer required epochs, and shorter effective training time compare to the custom CNN.

Analysis Learning Curves



The learning curves of cross-entropy loss and accuracy plotted against epoch for both models reveal several patterns.

ResNet-50 shows a rapid decrease in both training and validation loss, reaching very low values within a few epochs of Stage 2 fine-tuning. The loss then stabilises, showing that the model has quickly converged to a strong optimum. However, the custom CNN exhibits a more gradual decline in loss with small fluctuations.

The generalisation gap between training and validation accuracy remains relatively small for both models, indicating effective regularisation. For the custom CNN, minor gaps and fluctuations are still observed in later epochs. For ResNet-50, the gap is also minimal overall, only during Stage 2 the training accuracy occasionally reaches 100% while validation accuracy still remains slightly lower due to the model approaching the upper performance limit of the dataset.

In terms of training stability, the custom CNN shows more noticeable variability in validation accuracy across epochs, including dips such as around epochs 12 and 19. This illustrates how a smaller, from-scratch model is susceptible to batch-level fluctuations and sparse data. On the other hand, ResNet-50 exhibits validation curves that are far smoother and more stable, particularly during full fine-tuning. The robust pretrained feature representations, which only need small modifications during training, are the cause of this stability.

Part D: Analysis & Reflection

Analyse at least 10 misclassified samples. Propose one architectural or data improvement. Discuss the business application potential of your model in a Malaysian SME context.

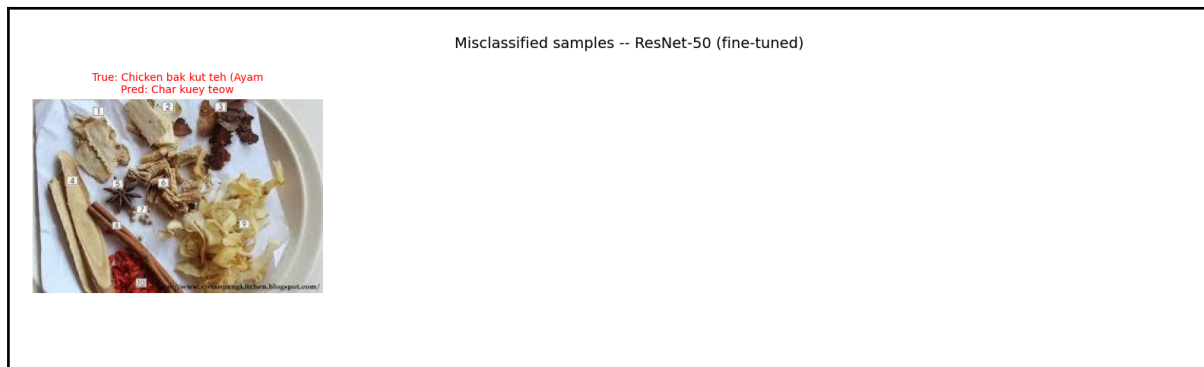
Misclassified Sample Evaluation

Misclassification analysis was performed on both custom CNN and ResNet-50 models. ResNet-50 produced only 1 misclassification on the 147-image test set while The custom CNN produced a total of 11 misclassifications.

Below is the breakdown of the errors of the misclassifications done for both models respectively:

ResNet-50:

Total misclassified: 1 / 147



True Class	Predicted Class	Count
Chicken Bak Kut Teh	Char kuey teow	1

For ResNet model, misclassifications are extremely rare. Only a single error is observed, where Chicken Bak Kut Teh is incorrectly predicted as Char Kuey Teow. This shows that ResNet model has strong feature extraction capability and high class separability, with minimal confusion between similar food categories.

Custom CNN:

Total misclassified: 11 / 147



True Class	Predicted Class	Count
Char kuey teow	Hong Kong Wonton noodles	2
Hong Kong Wonton noodles	Murtabak	2
Hong Kong Wonton noodles	Char kuey teow	1
Murtabak	Hong Kong Wonton noodles	3
Murtabak	Char kuey teow	1
Murtabak	Satay	1
Hong Kong Wonton noodles	Satay	1

Based on the wrong classification summary table, we can find out that the misclassification errors are concentrated in the custom CNN model. Hong Kong Wonton Noodles is the most frequently involved class appearing both as a true class and a predicted class in multiple misclassifications (4 times as true class and 5 times as predicted class). Similarly, Hong Kong Wonton Noodles is heavily involved in the misclassification, occurring as a true class in 5 cases and as a predicted class in 2 cases. This shows a strong bidirectional confusion between these classes and others. Classes like Satay and Bak Kut Teh show relatively fewer misclassifications, indicating more consistent predictions for these categories.

After reviewing all the misclassification, error types can be summarised as followed:

- strong visual similarity between classes, where dishes share similar textures, colors, and structures, such as different types of noodles or fried items.
- context bias by relying on background cues like plate type, banana leaves, or bowl presentation, which can mislead predictions when different classes share similar serving styles.
- High intra-class variation as the same dish can appear visually different due to variations in ingredients, preparation, and viewpoint.
- Atypical framing, such as close-up or partially visible dishes, reduces the availability of distinguishing features and makes classification more difficult.

Proposed Model Enhancement

The main issue of custom CNN is it gets confused by dishes that look visually similar like Char Kuey Teow and Hong Kong Wonton Noodles. Therefore, the targeted improvement is by replacing the standard Conv2d in the deeper blocks with Omni-Dimensional Dynamic Convolution (ODConv). In a standard convolution, the same fixed kernel is applied to all input images. ODConv generates custom, input-specific kernels by evaluating spatial and channel attention, allowing the network to zero in on the exact microscopic features that tell two similar dishes apart (Li et al., 2022).

This approach addresses the dominant error observed in the dataset, which is confusion caused by shared low-level features such as similar texture and colour. For example, when distinguishing Char Kuey Teow from Hong Kong Wonton Noodles, ODConv can emphasise the most discriminative features for each dish rather than relying on a single shared filter. Also, ODConv also introduces a small increase in parameters when applied selectively to the deeper layers, making it a more efficient improvement compared to increasing the overall

network size, making it a suitable and practical enhancement for improving classification performance in this task.

Application Considerations in SME Context

Target users:

small Malaysian F&B businesses like kopitiams, mamak stalls, food-court vendors, hawker stalls, and home-based meal businesses listed on Foodpanda and GrabFood. These businesses operate at thin margins and benefit most from low-cost automation.

Use cases:

- Calorie Tracking for Health Apps. Pairing the classifier with a Malaysian-food nutrition database like Malaysian Food Composition Database (MyFCD) (MyFCD, n.d.) lets users photograph a meal and receive an estimated calorie figure, which is useful for people with diabetes, fitness enthusiasts, and athletes in monitoring their dietary intake.
- Snap-and-Order Kiosks on Food Courts. Customers take photos of the dishes on display and the system identifies them and adds them to the order which helps to eliminate language barriers on menus
- Inventory and sales analytics for hawker stalls. A tablet camera photographs each plate going out and the system logs which dishes sell and when. Vendors can get an automatic log of what's selling fast and when and also useful for ingredient planning and reducing food waste.
- Auto-tagging for Foodpanda or GrabFood listings. Home-based businesses upload one photo and the system would instantly suggest the right dish category and tags. It takes the headache out of getting listed and keeps the apps clean and organized.
- Tourist's Food Guide Application. A tourist could point their phone at an unknown dish and instantly learn its name, a little bit of its backstory, the main ingredients, and crucial allergen warnings. Travellers can make more informed food choices while exploring local cuisine, especially when facing language barriers or unfamiliar dishes.

References

- Tahir, G. A., & Loo, C. K. (2020). *MF-150: Multilabel Malaysian food dataset*. IEEE DataPort. <https://ieee-dataport.org>
- Li, X., Wang, W., Hu, X., & Yang, J. (2022). *Omni-dimensional dynamic convolution*. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778).
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.
- Malaysian Food Composition Database (MyFCD). (n.d.). Nutrient composition of Malaysian foods. Retrieved from <https://myfcd.moh.gov.my>

Appendix

Dataset:

<https://drive.google.com/drive/folders/1iKM2peSh0m8V5jHIJYXiMmRp0AwQkP9c>

Code:

https://colab.research.google.com/drive/1CXR_xSsfPcAn3UWEY4eF-vq5KUa6gFo_

Github Repository:

<https://github.com/RextonRZ/Malaysian-Food-CNN-Classifier>